Lab. – AI-P3
Deep Learning in Production

A. Mhamdi

Dept. EE @ ISET Bizerte

You'll deploy a computer vision system capable of identifying everyday objects in images using the powerful `YOLO` architecture.

▼ What You'll Accomplish

This lab is all about seeing how it feels to deploy a real machine learning model:

1. *Explore the object detection dataset and understand the underlying data*
2. *Examine the* `YOLO` *model architecture and capabilities*
3. *Create a production-ready* `API` *endpoint using* `FastAPI`
4. *Embed the model on an ESP32 microcontroller*

1 Deploying a Computer Vision Model

In this lab, you will deploy a computer vision model capable of recognizing speed signs in Tunisia from images. This is a real-world application that could be used in a traffic management system to detect and warn drivers about speed limits.

1.1 Architecture

Deployment generally involves setting up all the necessary software for making predictions on a server. This setup allows clients to interact with the model by sending requests to the server. You can think of the server as a restaurant waiter: the client places an order (request), and the server responds by fulfilling the request, explaining options, or declining if it cannot be completed.

Model deployment is usually one of the final stages in the machine learning workflow. Here, we will use a pretrained object detection model known as YOLO, which is both fast and highly accurate, making it an excellent choice for this specific recognition task.

1.1.1 Server Side

The machine learning model is hosted on a server, where it waits to receive prediction requests from clients. To get a prediction, the client must supply all the necessary information the model requires. Often, clients will send multiple prediction requests together in a batch. The server processes these inputs and sends the resulting predictions back to the client, who can use them as needed. Communication between the client and server happens through the server's API, which defines how clients can interact with the server's functionality.

YOLO Model

YOLO *(You Only Look Once)* is a real-time object detection system that can identify and locate objects in images. YOLO is particularly efficient because it:

- Processes the entire image in a single pass
- Provides real-time detection capabilities
- Achieves good accuracy-speed trade-offs
- Can detect multiple object classes simultaneously

You can import the YOLO model (using `ultralytics` package) as follows:

```
from ultralytics import YOLO
# Load the YOLO model
model = YOLO('yolo.pt')
```

FastAPI Setup

FastAPI is a modern, fast web framework for building APIs with Python. It provides:

- Automatic API documentation generation
- Type hints support for better code quality
- High performance comparable to `Node.js`
- Easy integration with machine learning models

The main endpoint for object detection in FastAPI can be defined as follows:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class ImageRequest(BaseModel):
    image: str  # base64 encoded image data

@app.get("/")
def read_root():
    return {"message": "Hello, FastAPI!"}

@app.post("/detect")
async def detect_objects(request: ImageRequest):
    # The object detection logic here
    return {
        "objects": [],
        "processing_time": 0.0
    }
```

This endpoint will return a JSON response containing:

- Detected objects with bounding boxes
- Confidence scores for each detection
- Object class labels
- Processing time information

At last, you can run the FastAPI server as follows:

```
# Main function to run the FastAPI server
if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8000,␣
 ↪reload=True)
```

1.1.2 Client Side

Next, you'll write a simple client in Python that uses the requests library to communicate with your running FastAPI server. Make sure your server is up and

running before proceeding.

Here is a minimal working example (MWE) of how to send a GET request to your FastAPI server's root endpoint:

```
import requests

response = requests.get("http://localhost:8000/")
print(response.json())
```

This code sends a GET request to the server and prints the JSON response.

1.2 Terminology

1.2.1 HTTP Requests

The client and the server communicate with each other through a protocol called HTTP *(hypertext transfer protocol)*. This communication between client and server defines common actions called **requests** using verbs. Two very common requests are:

GET Retrieves information from the server.

POST Provides information to the server, which it uses to respond.

A GET request is used when your client wants to retrieve information from the server without sending any extra data—think of it as simply reading or viewing something. For example, when you visit a website's homepage, your browser sends a GET request to fetch and display the page.

On the other hand, a POST request is used when your client needs to send data to the server for processing. This is similar to submitting a form or posting a comment on social media: you provide information (like your comment), and the server processes it. For instance, if you want to upload an image for analysis, your client would send a POST request containing the image data to the server.

In the context of Machine Learning models hosted on a server, interactions are typically done via POST requests, because you need to supply the input data (such as an image or text) that the model will use to generate a prediction. For example, to get a prediction from an image classification model, you would send a POST request with the image data, and the server would respond with the predicted label.

1.2.2 Endpoints

A single server can host several different Machine Learning models. To distinguish between them, you assign a unique `endpoint` to each model. An endpoint is simply a specific pattern in the URL that tells the server which model to use. For instance, suppose your website is `jlai-p3.com`; you might organize your endpoints like this:

- `jlai-p3.com/detect-objects/`
- `jlai-p3.com/segment-image/`
- `jlai-p3.com/classify-animal/`

Each model would do what the name pattern suggests.

With `FastAPI`, you define an endpoint by first creating a function that will handle all of the logic for that endpoint. Then, you will use a decorator with a function that contains information about the HTTP method and URL pattern that will trigger your function.

2 Getting Started with Edge Impulse and ESP32

In this exercise, you will deploy a simple machine learning model to an ESP32 microcontroller using `Edge Impulse`. The goal is to make an LED blink faster or slower depending on the result of the model's inference[1]. This will help you understand how embedded ML works on real hardware.

2.1 Collect Data with Edge Impulse

2.2 Train a Simple Model

2.3 Deploy the Model to ESP32

2.4 Test Your System

Challenge: Because the ESP32 cannot handle computationally intensive models like YOLO, consider using it as an intermediary: have the ESP32 capture images (for example, using a camera module), transmit them to a more capable server running the YOLO model, and then forward the detection results back to the client. Try experimenting with sending images from the ESP32 and receiving real-time object detection feedback from your server!

[1] Think of a classification use case that you can solve with an ESP32

3 Conclusion

In this lab, you will gain hands-on experience deploying machine learning models in real-world scenarios. You will learn how to integrate `YOLO` for object detection with `FastAPI` to create scalable web services for computer vision applications. Additionally, you will use Edge Impulse to train and deploy a machine learning model directly onto an ESP32 microcontroller, enabling real-time inference on embedded hardware. This will give you practical skills in both cloud-based and edge AI deployment.